Static Members and Friends - C++ Program

```
#include <iostream>
#include <string>
using namespace std;

// Forward declaration of ClassB
class ClassB;

class ClassA {
private:
    int scoreA;

public:
    // Static data member
    static int objectCount;

    ClassA(int s = 0) : scoreA(s) {
        objectCount++;
    }

    // Static member function
    static int getObjectCount() {
        return objectCount;
    }

    // Friend function declaration
    friend void compareScores(const ClassA &a, const ClassB &b);

    // Friend class declaration
    friend class Inspector;
};

// Definition of static member variable
int ClassA::objectCount = 0;

class ClassB {
private:
    int scoreB;

public:
    ClassB(int s = 0) : scoreB(s) {}

    // Friend function declaration
    friend void compareScores(const ClassA &a, const ClassB &b);
};
```

Made with Julius

Static Members and Friends - C++ Program

```
// Friend function definition accessing private data of ClassA and ClassB
void compareScores(const ClassA &a, const ClassB &b) {
    cout << "\n[Friend Function] Comparing Private Data:\n";
    cout << " ClassA Score: " << a.scoreA << "\n";
    cout << " ClassB Score: " << b.scoreB << "\n";
    if (a.scoreA > b.scoreB) {
        cout << " Result: ClassA has a higher score.\n";
    } else if (b.scoreB > a.scoreA) {
        cout << " Result: ClassB has a higher score.\n";
    } else {
        cout << " Result: Both scores are equal.\n";
    }
}

// Friend Class Inspector accessing private members of ClassA
class Inspector {
public:
    void inspectClassA(const ClassA &a) const {
        cout << "\n[Friend Class Inspector] Private scoreA of ClassA = "
             << a.scoreA << endl;
    }
};

int main() {
    cout << "=== Demonstration 1: Static Data Member & Member Function ===\n";
    ClassA obj1(85), obj2(92), obj3(78);
    cout << "Total ClassA objects created: " << ClassA::getObjectCount() << "\n";
    cout << "\n=== Demontration 2: Friend Function ===";
    compareScores(obj2, objB);
    cout << "\n=== Demonstration 3: Friend Class ===";
    Inspector inspectorObj;
    inspectorObj.inspectClassA(obj2);
    return 0;
}
```

Made with Julius